UROOM ANALYTICS

Documentation Technique — Intégration API

Guide complet pour le développeur backend

Version 1.0 — Mars 2026

1. Vue d'ensemble

L'API Uroom Analytics est un service Python (FastAPI) qui expose le modèle de Machine Learning de prédiction de potentiel de jeunes joueurs de football. Le backend Node.js/Express communique avec cette API via des requêtes HTTP REST.

Architecture globale

Web App (React/Next.js)

↓ HTTP

Backend Node.js / Express ← ton code existant

↓ HTTP POST/GET

API FastAPI Python (port 8000) ← nouveau service

↓

Modèle ML + uroom/ (radar, features, similarity...)

Ce que l'API fait

- Calcule le score de potentiel ML (0-100%) pour un joueur
- Génère les 8 scores du radar (Vitesse, Dribble, Tir, Passe, Défense, Physique, Contrôle, Vision)
- Projette l'évolution du joueur sur 1 à 4 ans
- Retourne les alertes scout et indicateurs de fiabilité
- Supporte les requêtes batch pour scorer une équipe entière

2. Démarrage de l'API

Prérequis

Outil	Version	Installation
Python	3.11+	python3 --version
FastAPI	0.100+	pip install fastapi
Uvicorn	0.20+	pip install uvicorn
Modèle entraîné	uroom_model_v2.pkl	data/processed/

Installation

```
cd uroom-ia
pip install fastapi uvicorn
```

Structure des fichiers

```
uroom-ia/
■■■ api/
■   ■■■ __init__.py       ← fichier vide obligatoire
■   ■■■ main.py           ← code de l'API
■■■ uroom/
■   ■■■ radar.py
■   ■■■ features.py
■   ■■■ model.py
■   ■■■ similarity.py
■   ■■■ data_loader.py
■   ■■■ schemas.py
■■■ data/
    ■■■ processed/
        ■■■ uroom_model_v2.pkl   ← modèle entraîné
```

Lancer l'API

```
cd uroom-ia
uvicorn api.main:app --reload --port 8000
```

Vérification : ouvre http://localhost:8000/docs dans le navigateur. Tu verras l'interface Swagger interactive avec tous les endpoints.

Variable d'environnement dans Node.js

```
UROOM_API_URL=http://localhost:8000
```

3. Endpoints de l'API

GET /health — Vérification

Vérifie que l'API et le modèle sont chargés. À appeler au démarrage de Node.js.

```
{ "status": "ok", "model_loaded": true, "n_features": 11, "threshold": 0.5 }
```

POST /predict — Prédiction joueur

Endpoint principal. Envoie les snapshots d'un joueur et reçoit son score ML complet avec radar et projections.

Corps de la requête (JSON)

```
{
  "snapshots": [
    {
      "player_id":         "P0025",
      "birth_date":        "2010-05-15",
      "position":          "MID",
      "dominant_foot":     "right",
      "nationality":       "BE",
      "academy":           "Anderlecht Academy",
      "is_pro":            null,
      "snapshot_date":     "2024-05-15",
      "season":            "2023-24",
      "age":               13.9,
      "competition_level": 2,
      "height_cm":         162.5,
      "weight_kg":         52.0,
      "sitting_height_cm": 85.0,
      "sprint_10m_sec":    1.94,
      "sprint_30m_sec":    4.52,
      "jump_cm":           38.5,
      "endurance_score":   54.0,
      "matches_played":    15,
      "minutes_played":    1200,
      "pass_accuracy_pct": 72.5,
      "dribble_success_pct": 58.0,
      "shots_on_target_pct": 42.0,
      "duels_won_pct":     51.0,
      "interceptions_per90": 2.3,
      "goals_per90":       0.2,
```

```
      "assists_per90":     0.15,
      "avg_scout_score":   7.5,
      "scout_observation_count": 2
    }
  ],
  "years_ahead": 4
}
```

Réponse

```
{
  "player_id":        "P0025",
  "potential_score":  77.8,
  "verdict":          "Tres prometteur",
  "score_reliability": "high",
  "scout_alert":      null,
  "scout_coverage":   2,
  "radar": {
    "vitesse": 74, "dribble": 68, "tir": 71,
    "passe": 79, "defense": 65, "physique": 72,
    "controle": 73, "vision": 58
  },
  "projections": {
    "2024-25": {
      "scores": { "vitesse": 77, "dribble": 72, ... },
      "avg_improvement": 4.2,
      "display_message": "Prediction IA Uroom (2024)...",
      "is_projected": true
    },
    ...
  },
  "timeline": { /* history + projections fusionnes */ }
}
```

POST /predict/batch — Prédiction équipe

Score plusieurs joueurs en une requête. Retourne un tableau trié par potential_score décroissant.

```
{
  "players": [
    { "snapshots": [ { ...snapshot joueur 1... } ] },
    { "snapshots": [ { ...snapshot joueur 2... } ] }
  ],
  "years_ahead": 4
}
```

4. Intégration Node.js / Express

Service uroomAI.js

Place ce fichier dans services/uroomAI.js dans ton projet Node.js :

```
const UROOM_API_URL = process.env.UROOM_API_URL || 'http://localhost:8000'

async function predictPlayerPotential(snapshots, yearsAhead = 4) {
  const response = await fetch(`${UROOM_API_URL}/predict`, {
    method:  'POST',
    headers: { 'Content-Type': 'application/json' },
    body:    JSON.stringify({ snapshots, years_ahead: yearsAhead }),
  })
  if (!response.ok) {
    const error = await response.json()
    throw new Error(`Uroom API error: ${error.detail}`)
  }
  return response.json()
}
module.exports = { predictPlayerPotential }
```

Route Express exemple

```
const { predictPlayerPotential } = require('../services/uroomAI')

router.post('/players/:id/predict', async (req, res) => {
  try {
    const { snapshots } = req.body
    const result = await predictPlayerPotential(snapshots)
    res.json({
      potential_score: result.potential_score,
      verdict:         result.verdict,
      radar:           result.radar,
      projections:     result.projections,
    })
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
})
```

5. Format des données

Champs obligatoires par snapshot

Champ	Type	Description	Exemple
player_id	string	ID unique du joueur	P0025
birth_date	string	Date naissance ISO	2010-05-15
position	string	MID / DEF / FWD / GK	MID
dominant_foot	string	right / left / both	right
snapshot_date	string	Date de la mesure ISO	2024-05-15
season	string	Saison	2023-24
age	float	Age décimal	13.9
competition_level	int	1=national élite, 5=district	2

Champs physiques

Champ	Type	Description	Défaut si absent
height_cm	float	Taille debout (cm)	0.0
weight_kg	float	Poids (kg)	0.0
sitting_height_cm	float	Hauteur assise (cm) — PHV	0.0
sprint_10m_sec	float	Sprint 10m (secondes)	0.0
sprint_30m_sec	float	Sprint 30m (secondes)	0.0
jump_cm	float	Détente verticale (cm)	0.0
endurance_score	float	Score Yo-Yo / navette	0.0

Champs techniques (match ou atelier)

Champ	Type	Description	Défaut
matches_played	int	Matchs joués	0
minutes_played	int	Minutes jouées	0
pass_accuracy_pct	float	% passes réussies	0.0

Champ	Type	Description	Défaut
dribble_success_pct	float	% dribbles réussis	0.0
shots_on_target_pct	float	% tirs cadrés	0.0
duels_won_pct	float	% duels gagnés	0.0
interceptions_per90	float	Interceptions par 90 min	0.0
goals_per90	float	Buts par 90 min	0.0
assists_per90	float	Passes décisives par 90 min	0.0

Champs scout (optionnels)

Champ	Type	Description
avg_scout_score	float | null	Note moyenne scout (1-10)
scout_observation_count	int	Nombre d'observations scout
is_pro	int | null	1=pro, 0=non-pro, null=inconnu

Important — Champs manquants

Si un champ physique ou technique est inconnu, envoyer 0.0.

Le pipeline gère les valeurs manquantes via imputation médiane.

Le modèle fonctionne avec des données partielles.

La sitting_height_cm est importante pour le calcul PHV.

Sans elle, le système bascule sur une méthode simplifiée.

6. Les 8 dimensions du radar

Dimension	Source principale	Calcul
Vitesse	sprint_30m_sec	Inversé + normalisé par âge
Dribble	dribble_success_pct	Normalisé par âge (percentile)
Tir	shots_on_target_pct	Normalisé + bonus goals_per90
Passe	pass_accuracy_pct	Ajusté par niveau de compétition
Défense	duels_won_pct + interc.	70% duels + 30% interceptions
Physique	endurance + jump_cm	Z-scores PHV par stade biologique
Contrôle	passe + dribble	Moyenne Passe/Dribble ajustés
Vision	assists + goals per90	60% assists + 40% goals

Tous les scores sont normalisés 0-100 en comparant le joueur à ses pairs du MÊME stade de maturité biologique (PHV offset) — pas juste du même âge. C'est la correction du biais de maturité.

Projections futures

Les projections 1-4 ans sont personnalisées selon 3 facteurs :

- Pente personnelle : régression linéaire sur les snapshots passés du joueur
- Taux normatif par âge : un joueur de 12 ans progresse naturellement plus vite qu'un joueur de 16 ans
- Score ML de potentiel : un joueur à 80% progresse 1.6x plus vite qu'un joueur à 20%

7. Gestion des erreurs

Code HTTP	Cause	Action
200	Succès	Utiliser la réponse normalement
400	Snapshots manquants	Vérifier que snapshots[] n'est pas vide
422	Features impossibles	Vérifier le format des champs obligatoires
503	Modèle non chargé	Attendre ou redémarrer l'API
500	Erreur interne	Logger error.detail et contacter l'équipe

Pattern recommandé dans Node.js

```
try {
  const result = await predictPlayerPotential(snapshots)
  // utiliser result
} catch (error) {
  console.error('Uroom API error:', error.message)
  // fallback : afficher scores a 0 ou message d'erreur
}
```

8. Checklist d'intégration

Étape 1 — Préparer l'environnement Python

- [ ] Cloner le repo uroom-ia sur le serveur
- [ ] pip install fastapi uvicorn
- [ ] Vérifier que data/processed/uroom_model_v2.pkl existe
- [ ] Créer api/__init__.py (fichier vide)
- [ ] Copier api/main.py

Étape 2 — Lancer et tester l'API

- [ ] uvicorn api.main:app --reload --port 8000
- [ ] GET http://localhost:8000/health → status: ok
- [ ] Ouvrir http://localhost:8000/docs et tester /predict manuellement

Étape 3 — Intégrer dans Node.js

- [ ] Copier services/uroomAI.js dans le projet Node
- [ ] Ajouter UROOM_API_URL=http://localhost:8000 dans .env
- [ ] Créer la route POST /api/players/:id/predict
- [ ] Tester depuis Postman ou le frontend

Étape 4 — Brancher le frontend

- [ ] radar.scores → alimenter le composant radar chart
- [ ] radar.timeline → alimenter le sélecteur d'années
- [ ] projections[saison].display_message → texte sous le radar
- [ ] potential_score + verdict → carte joueur

Contact & support

Pour toute question sur le modèle ou l'API :

→ Consulter ce document en priorité

→ Tester via http://localhost:8000/docs (Swagger UI)

→ Les logs de l'API affichent les erreurs détaillées dans le terminal